

UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS DE CRATEÚS

Sistemas Distribuídos
Professor: Anderson Almada

Segunda Prática

A atividade tem como objetivo praticar a implantação e o gerenciamento de uma aplicação containerizada utilizando **Kubernetes**. Nesta prática, será utilizada a mesma aplicação desenvolvida na **Primeira Prática**, que consiste em uma aplicação Flask responsável por contabilizar o número de acessos utilizando o Redis. Não é necessário desenvolver novamente a aplicação, criar o Dockerfile ou utilizar Docker Compose. A aplicação da primeira prática será utilizada como ponto de partida.

1. O problema

Na primeira prática, a aplicação foi executada utilizando Docker Compose:

Navegador → Flask/App → Redis → Volume

Agora, imagine que a aplicação esteja crescendo e seja necessário:

- executar várias instâncias da aplicação;
- distribuir as requisições entre essas instâncias;
- recuperar automaticamente containers que apresentem falhas;
- aumentar ou diminuir o número de instâncias;
- realizar atualizações sem interromper a aplicação.

Para isso, a aplicação será implantada em um **cluster Kubernetes**.

2. Prepare o ambiente

Após instalado o minikube, execute o seguinte comando:

- minikube start

Questão 1: O que representa o nó apresentado pelo comando kubectl get nodes?

3. Preparar a imagem da aplicação

A aplicação Flask já foi desenvolvida na primeira prática. Agora, a imagem da aplicação deverá estar disponível dentro do ambiente do Minikube. A partir do diretório do projeto, execute:

- minikube image build -t contador:1.0 .

Verifique se a imagem foi criada:

- minikube image ls | grep contador

A imagem contador:1.0 será utilizada pelo Kubernetes para executar os Pods da aplicação.

4. Criar o deployment da aplicação

Crie um diretório chamado:

k8s/

Dentro dele, crie o arquivo:

app-deployment.yaml

O Deployment deverá executar **3 réplicas** da aplicação Flask.

Ele deverá:

- utilizar a imagem contador:1.0;
- criar 3 Pods;
- configurar a variável REDIS_HOST;
- utilizar o nome redis para localizar o Redis.

Questão 2: Por que a aplicação deve utilizar redis em REDIS_HOST, em vez do endereço IP de um Pod Redis?

4. Criar o deployment do Redis

Crie o arquivo:

redis-deployment.yaml

O Redis deverá ser executado pelo Kubernetes.

Nesta etapa, utilize:

- imagem redis:latest;
- 1 réplica;

Questão 3: Por que a aplicação possui 3 réplicas enquanto o Redis possui apenas 1 réplica nesta prática?

5. Criar um service para o Redis

Os Pods podem ser recriados pelo Kubernetes e, consequentemente, seus endereços IP podem mudar. Para fornecer um ponto de acesso estável ao Redis, crie um **Service**.

Crie:

redis-service.yaml

O Service deverá:

- ser chamado redis;
- utilizar a porta 6379;
- encaminhar as requisições para os Pods que possuem a label: app: redis

Questão 4: Qual é a função do Service do Redis?

6. Criar um service da aplicação

Agora é necessário disponibilizar a aplicação Flask para acesso externo.

Crie:

app-service.yaml

O Service deverá:

- ser chamado contador;
- utilizar o tipo NodePort;
- utilizar a porta 5050;
- encaminhar as requisições para os Pods da aplicação;

- utilizar a label: app: contador

Questão 5: Como o Service contador sabe quais Pods devem receber as requisições?

7. Executar a aplicação

Aplique todos os recursos Kubernetes:

- kubectl apply -f k8s/

Verifique os Pods:

- kubectl get pods

Verifique os Deployments:

- kubectl get deployments

Verifique os Services:

- kubectl get services

Você deverá observar:

- **3 Pods** da aplicação;
- **1 Pod** do Redis;
- **2 Deployments (aplicação e redis)**;
- **2 Services (aplicação e redis)**.

Para acessar a aplicação pelo navegador:

- minikube service contador --url

Abra o endereço apresentado pelo comando. O contador deverá funcionar da mesma forma que na primeira prática.

8. Escalar a aplicação

Na primeira prática, havia apenas uma instância da aplicação. Agora, o Kubernetes deverá executar múltiplas réplicas.

Aumente o número de réplicas para 5:

- kubectl scale deployment <nome do deployment do contador> --replicas=5

Verifique novamente:

- kubectl get pods

Agora reduza para 2:

- kubectl scale deployment <nome do deployment do contador> --replicas=2

Verifique novamente:

- kubectl get pods

Questão 6: O que aconteceu com os Pods quando o número de réplicas foi alterado de 3 para 5 e depois para 2?

9. Self-Healing

O Kubernetes pode detectar quando o estado atual da aplicação não corresponde ao estado desejado.

Escolha um dos Pods da aplicação e remova-o:

- kubectl delete pod <nome-do-pod>

Verifique novamente:

- kubectl get pods

Questão 7: O Pod removido foi simplesmente perdido? Explique o que aconteceu após sua exclusão.

10. Rolling Update

Agora será realizada uma alteração na aplicação. Modifique a mensagem apresentada pela aplicação. Por exemplo:

Olá, Kubernetes!

Reconstrua a imagem com uma nova versão:

- minikube image build -t contador:2.0 .

Altere o Deployment para utilizar: contador:2.0

Aplique novamente o manifesto:

- kubectl apply -f k8s/app-deployment.yaml

Acompanhe a atualização:

- kubectl rollout status deployment/contador-deployment

Também é possível observar os Pods:

- kubectl get pods

E verificar o histórico:

- kubectl rollout history deployment/contador-deployment
- kubectl rollout history deployment/contador-deployment --revision=2

Questão 8: O que caracteriza um Rolling Update? Por que ele é útil em aplicações que precisam permanecer disponíveis durante uma atualização?

11. Rollback

Agora simule uma situação em que a nova versão apresenta algum problema.

Utilize:

- kubectl rollout undo deployment/contador-deployment

Verifique o resultado:

- kubectl rollout status deployment/contador-deployment

Confira novamente a aplicação no navegador.

Questão 9: Qual é a função do comando kubectl rollout undo?

12. Entrega

Além das respostas a todas as perguntas, entregar o projeto em formato zip ou repositório (github) e um vídeo explicando todo o funcionamento (aplicação funcionando; Pods em execução; Deployments; Services; Escalabilidade; Self-Healing; Rolling Update; Rollback)